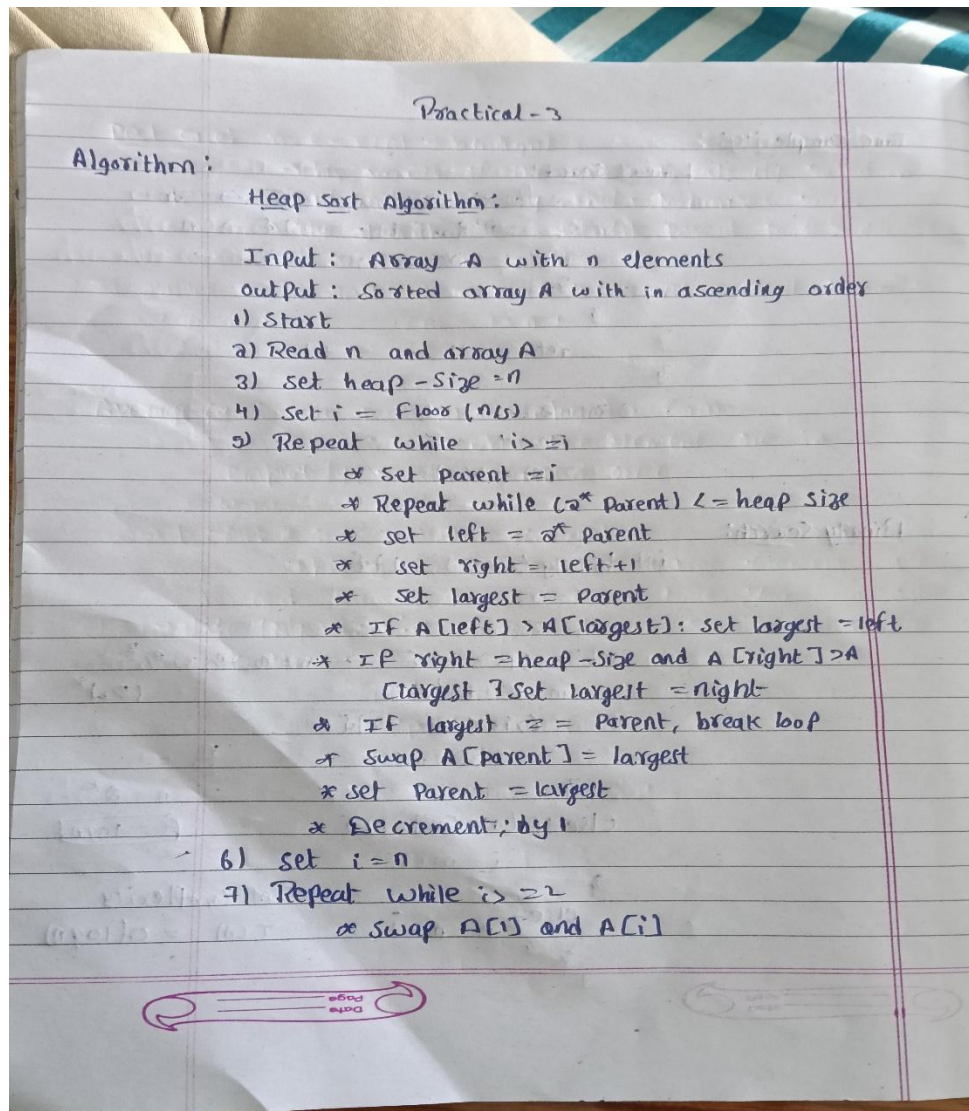


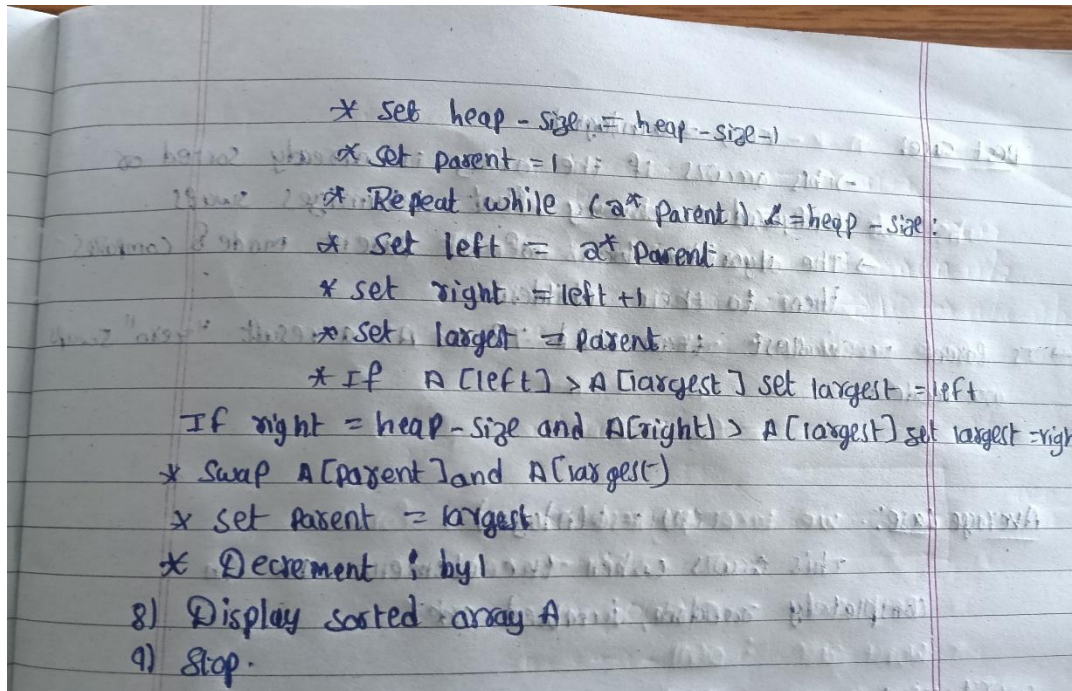


PRACTICAL- 3

AIM : To Implementation of Max-Heap sort algorithm

ALGORITHM :





Program / Code :

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;

    int left = 2 * i + 1;

    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
```



```
{  
    int temp = arr[i];  
    arr[i] = arr[largest];  
    arr[largest] = temp;  
    heapify(arr, n, largest);  
}  
}  
void heapSort(int arr[], int n)  
{  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(arr, n, i);  
    for (int i = n - 1; i > 0; i--)  
    {  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
        heapify(arr, i, 0);  
    }  
}  
int main()  
{  
    int n;  
    int arr[100];
```



```
cout << "Enter the number of elements: ";  
  
cin >> n;  
  
cout << "Enter " << n << " elements:\n";  
  
for (int i = 0; i < n; i++)  
{  
    cin >> arr[i];  
}  
  
cout << "\nOriginal array: ";  
  
for (int i = 0; i < n; i++)  
{  
    cout << arr[i] << " ";  
}  
  
heapSort(arr, n);  
  
cout << "\nSorted array: ";  
  
for (int i = 0; i < n; i++)  
{  
    cout << arr[i] << " ";  
}  
  
cout << endl;  
  
return 0;  
}
```

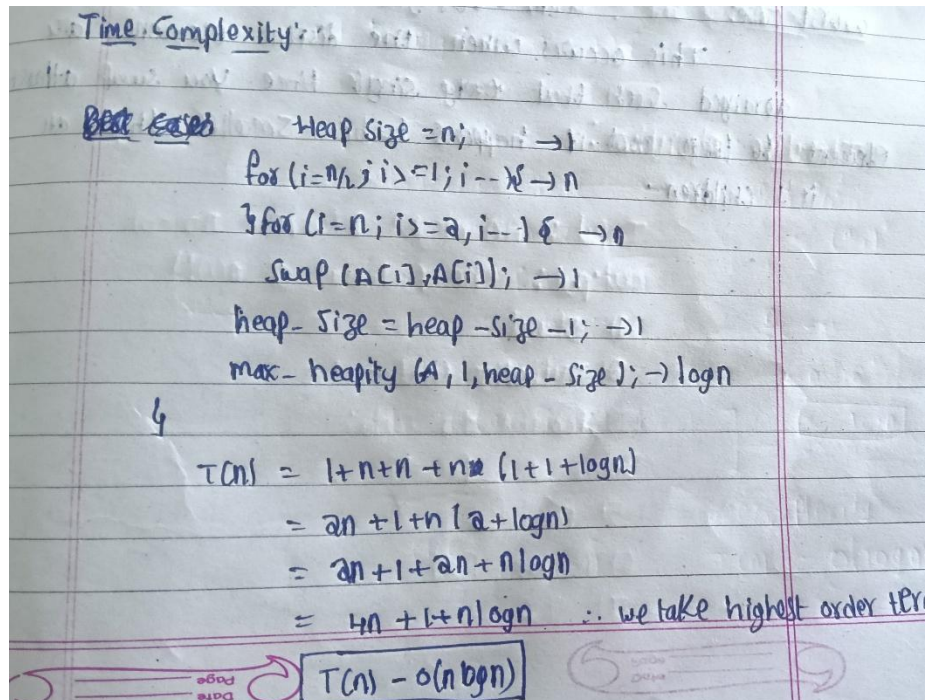


Output :

```
Enter the number of elements: 5
Enter 5 elements:
12 23 3 16 7

Original array: 12 23 3 16 7
Sorted array: 3 7 12 16 23
```

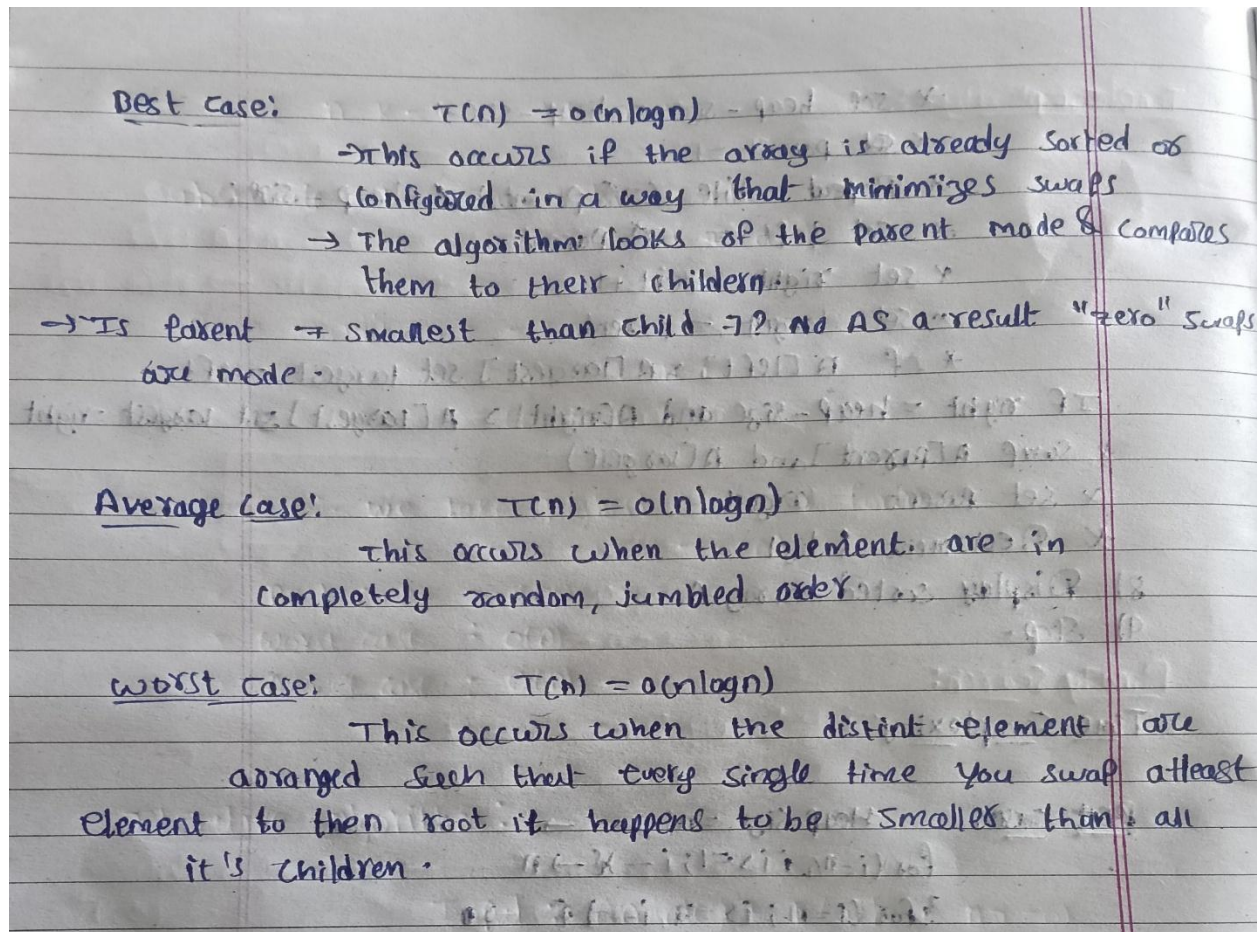
Time Complexity :



Best case : $O(n \log n)$

Average case : $O(n \log n)$

Worst case : $O(n \log n)$



Conclusion : The Max-Heap Sort algorithm was successfully implemented and executed to sort the elements in ascending order. The program verified a consistent time complexity of ($O(n \log n)$) across the best, average, and worst cases using an in-place binary heap structure.